

Reviewer Pack — Minimal Rank of Categorified Boolean Algebras for Monotone k...

Entry 00dc2e6a4fb8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 11:14:52 UTC

Minimal Rank of Categorified Boolean Algebras for Monotone k-CLIQUE Proofs

Entry ID: 00dc2e6a4fb8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 11:14:52 UTC

1. Conjecture

Field A (mathematical branch): Categorification Theory **Field B** (complexity object): Monotone Circuit Complexity

Statement:

[‘For every monotone CNF formula F representing k -CLIQUE, the minimal rank of its categorified Boolean algebra is $\Theta(n^k)$.’, ‘The categorified Boolean algebra of a CNF F is constructed using the formalism of monoidal categories and is defined as the free completion of the Boolean algebra generated by F .’, ‘This statement implies that there exists no polynomial-size monotone circuit for k -CLIQUE.’]

Rationale (proposer’s reasoning):

[‘Categorification Theory provides a general framework to study structures through higher categorical structures, which could potentially reveal hidden complexities in boolean algebras.’, ‘By applying categorification to Boolean algebras, we may uncover invariants that are not apparent at the first order of abstraction, possibly explaining the non-existence of polynomial-size monotone circuits for k -CLIQUE.’, ‘The connection between categorified structures and computational complexity has been explored in other contexts but remains largely unexplored in the context of monotone circuit lower bounds.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 852983fd55f53006

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal rank of categorified Boolean algebras for monotone k -CLIQUE instances is within $\Theta(n^k)$ with a correlation coefficient ≥ 0.8 and p -value ≤ 0.05 across 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	UNCERTAIN	1.00	HITS	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - categorification theory AND monotone k-clique proofs - monotone circuit complexity AND categorified Boolean algebra rank $\Theta(n^k)$ - free completion monoidal categories AND Boolean algebra k-CLIQUE

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.3s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j

        # Swap rows
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]

        # Eliminate below pivot
        for j in range(i+1, n):
            factor = -A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] += factor * A[i][k]
            b[j] += factor * b[i]

    # Back-substitute
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]

    return x
```

```

def matrix_multiply(A, B):
    m, k = len(A), len(B[0])
    result = [[0] * k for _ in range(m)]
    for i in range(m):
        for j in range(k):
            for l in range(len(B)):
                result[i][j] += A[i][l] * B[l][j]
    return result

def rank(A):
    n, m = len(A), len(A[0])
    A_augmented = [row + [1 if i == j else 0 for j in range(m)] for i, row in enumerate(A)]
    _, b = gaussian_elimination(A_augmented, [0] * n)
    return sum(1 for x in b if x != 0)

def generate_k_clique_cnf(n, k):
    variables = list(range(n))
    clauses = []
    for subset in itertools.combinations(variables, k):
        clause = [-x-1 for x in subset]
        clauses.append(clause)
    return clauses

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [30, 35, 40]
    results = []

    for n in n_values:
        k = min(n // 2, 5) # Ensure k is at least 1
        cnf = generate_k_clique_cnf(n, k)

        rank_sum = 0
        expected_rank = sum(2**i for i in range(k))

        for _ in range(30):
            A = [[0] * n for _ in range(n)]
            b = [0] * n

            for clause in cnf:
                for x in clause:
                    if x > 0:
                        A[x-1][x-1] += 1
                    else:
                        A[-x-1][-x-1] += 1

            rank_sum += rank(A)

        mean_rank = rank_sum / 30
        results.append({
            "n": n,
            "rank": mean_rank,
            "expected_rank": expected_rank
        })

```

```

correlation_coefficient = (sum((result["rank"] - sum(result["rank"] for result in results) / len(results))
                             * (result["rank"] - sum(result["expected_rank"] for result in results) / len(results))
                             for result in results) / len(results))

if correlation_coefficient >= 0.8:
    conjecture_holds = True
else:
    conjecture_holds = False

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_d = sum(result["metric_value"] for result in results) / len(results)
    std_dev = math.sqrt(sum((result["metric_value"] - mean_d)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_d} std={std_dev} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6b27639c.py", line 126, in <module>

```

    trial_result = run_trial(seed)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6b27639c.py", line 92, in run_trial

```

    rank_sum += rank(A)

```

^^^^^^

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6b27639c.py", line 57, in rank
_, b = gaussian_elimination(A_augmented, [0] * n)
^^^^
```

ValueError: too many values to unpack (expected 2)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not meet the pre-registered support condition and thus cannot be confirmed as SUPPORTED. | next: Re-run the test with proper error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11562
2	preregistration	ollama_remote	glm4:latest	0	0	5331
3	novelty	ollama_remote	glm4:latest	0	0	4582
4	novelty	ollama_remote	glm4:latest	0	0	5489
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12180
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9002
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8856
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13807
9	judge	ollama_remote	glm4:latest	0	0	15248

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 86057 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/00dc2e6a4fb8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/00dc2e6a4fb8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/00dc2e6a4fb8.tar.gz` (if generated)